

Building Cryptographic Proofs: Zero-Knowledge Proofs & SNARKs

Final Project – Extending/Finding a SNARK Application from the Literature

39th Escuela de Ciencias Informáticas (ECI), University of Buenos Aires – July 2026

1. Overview

The final evaluation of the course is a project in which you will select a research paper that presents an **application of SNARKs**, published at a cryptography or applied-cryptography venue, and **propose and (partially) implement a motivated extension** of its contribution.

The goal is to reproduce, in a small scale, the workflow of applied cryptographic research: understand an existing system, identify its limitations, argue which improvements are feasible, implement a meaningful subset of them, and measure the result honestly.

Otherwise, you understand how a solution works and then you find a valuable application of this solution.

2. Timeline

- **Day 0 (last day of the course):** this project description and the list of eligible papers are released.
- **August 7th:** deadline to propose a group and a project (see Section 4).
- **September 10th (approximately 4 weeks):** project submission deadline.
- **At most October 20th:** grading and feedback.

3. Team

The project should be carried out **in groups of 2-3 people**.

Groups submit a single repository. The README must include a short statement of who did what.

4. Paper Selection and Reservation

You can choose one paper from the curated list in Section 8.

Within one week of the release of this document, submit **a response to the form describing which are the components of your group and the starting paper you chose for your take-home exam**.

<https://forms.gle/MBjARUAKrpfIFaWw9>

5. Deliverable

The deliverable is a **public GitHub repository** containing:

- A **README** (the main written report) answering the questions in Section 6. The README should contain at most **1500 words**.
- The **source code** of your implementation, with clear build/run instructions so that results are reproducible.
- The **benchmark scripts and raw measurements** used to produce your performance numbers.
- It is **recommended** to set up Continuous Integration (e.g., [GitHub Actions](#)) running a unit test suite that demonstrates **completeness** (a valid witness produces a proof accepted by the verifier) and **soundness** (an invalid witness is correctly rejected) of your implementation.
- You can optionally add to the repository a link to a **video** with a **demo** of your **work**.

Submit the repository URL via **Google Classroom** by the deadline.

The repository state at the deadline (last commit before the deadline) is what will be graded.

6. Required Content (README Structure)

Your README must answer, at minimum, the following questions:

1. **The application.** What problem does the paper solve, and what role does the zero-knowledge proof play in the system? What exactly is being proven, by whom, and to whom? What are the statement, the witness, and the trust assumptions?
2. **The proof system and framework.** Which SNARK (or proof system) does the paper use, and which framework/library implements it? What are the relevant properties (transparent vs. trusted setup, proof size, verifier cost, recursion support, etc.)?
3. **What can be improved/On what the solution can be used.** Identify limitations of the paper's construction or implementation (e.g., prover time, proof size, expressiveness of the statement, deployment constraints, security assumptions) and propose one or more concrete extensions, with a clear motivation for each.
4. **Feasibility analysis.** For each proposed extension, argue whether it is feasible within the scope of this project and why. Distinguish explicitly between improvements that are feasible but out of scope, improvements that are open research problems, and improvements you actually implemented.
5. **Your implementation.** What did you implement, using which framework, and why that framework? Describe the design and any deviations from your original plan. How did you test the correctness of your implementation (completeness and soundness)? If you set up a CI pipeline, include a link/badge to it.
6. **Performance.** Report the performance of your implementation (at least: proving time, verification time, proof size; specify the hardware used). Compare against a **baseline**: the numbers reported in the original paper (if available and comparable) and/or a naive baseline you define and justify. Isolated numbers without a comparison point will not be considered meaningful.

7. Grading Rubric

- Understanding (1) of the paper, (2) of the role of the zk-proof and implementation (questions 1–2).
- Quality and originality of the proposed extension(s), and strength of the motivation (question 3).
- Feasibility analysis: correctness and honesty of the assessment (question 4).
- Implementation: correctness, scope, and code quality (question 5) — including whether tests (ideally run via CI) demonstrate completeness/soundness, not just absence of runtime errors.
- Benchmarking rigor: methodology, baseline comparison, reproducibility (question 6).
- Clarity and organization of the repository and README.

Academic integrity. You may use any tool, including AI assistants, but you are fully responsible for everything in your repository and you must be able to explain and defend every design choice and every line of code. Reusing third-party code is allowed only with clear attribution.

8. Eligible Papers

#	Title	Authors	Venue	Year	Link	Notes
1	Zero-Knowledge Middleboxes	P. Grubbs, A. Arun, Y. Zhang, J. Bonneau, M. Walfish	USENIX Security	2022	ePrint	
2	Trust Nobody: Privacy-Preserving Proofs for Edited Photos with Your Laptop	P. Della Monica, I. Visconti, A. Vitaletti, M. Zecchini	IEEE S&P	2025	ePrint	
3	Decentralized Fair Exchange with Advertising	P. Della Monica, I. Visconti, A. Vitaletti, M. Zecchini	CANS	2025	Springer	Using it in different use cases rather than images is ok also
4	ACTS: Attestations of Contents in TLS Sessions	P. Della Monica, I. Visconti, A. Vitaletti, M. Zecchini	NDSS	2026	NDSS	
5	Reckle Trees: Updatable Merkle Batch Proofs with Applications	C. Papamanthou, S. Srinivasan, N. Gailly, I. Hishon-Rezaizadeh, A. Salumets, S. Golemac	ACM CCS	2024	ePrint	Using it in different use cases exploiting mapreduce paradigm is fine also
6	Efficient Query Verification for Blockchain Superlight Clients Using SNARKs	S. De Angelis, I. Visconti, A. Vitaletti, M. Zecchini	Blockchain: Research and Applications	2025	ScienceDirect	

7	ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs	B.-J. Chen, S. Waiwitlikhit, I. Stoica, D. Kang	EuroSys	2024	ACM DL	For using Halo2 try the following tutorial: https://halo2.zksecurity.xyz
8	zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure	M. Rosenberg, J. White, C. Garman, I. Miers	IEEE S&P	2023	ePrint	
9	zkLogin: Privacy-Preserving Blockchain Authentication with Existing Credentials	F. Baldimtsi, K. Chalkias, Y. Ji, J. Lindstrøm, D. Maram, B. Riva, A. Roy, M. Sedaghat, J. Wang	ACM CCS	2024	arXiv	
10	zkRelay: Facilitating Sidechains using zkSNARK-based Chain-Relays	M. Westerkamp, J. Eberhardt	IEEE EuroS&PW	2020	IEEE	
11	Kryvos: Publicly Tally-Hiding Verifiable E-Voting	N. Huber, R. Küsters, T. Krips, J. Liedtke, J. Müller, D. Rausch, P. Reisert, A. Vogt	ACM CCS	2022	ePrint	
12	Election Eligibility with OpenID: Turning Authentication into Transferable Proof of Eligibility	V. Cortier, A. Debant, A. Goetschmann, L. Hirschi	USENIX Security	2024	USENIX	
13	VIMz: Private Proofs of Image Manipulation using Folding-based zkSNARKs	S. Dziembowski, S. Ebrahimi, P. Hassanizadeh	PoPETs	2025	ePrint	Explain how it differs from [2] and design the solution properly

You can also propose other papers, but please motivate your choice during the form submission.

9. Contact

Contact me by email at zecchini@diag.uniroma1.it